Evidencias — Fase 2: la arquitectura corre

Proyecto: Consulta de stock de una cadena de farmacias
Autor: Sebastian Hoffmann
Repositorio: https://github.com/hoffmannnactochsebastianalexan-ux/stock-farmacias-isi601 (etiqueta fase-2)

Todo se levanta con un solo comando (ver README):

```
docker compose up -d --build
```

> En Windows PowerShell usa `curl.exe` (no `curl`, que en PowerShell 5.1 es otro comando). Los comandos `docker compose exec` se ejecutan desde la carpeta del proyecto.

0. Todo levantado

```
docker compose ps
SERVICE       STATUS                    PORTS
app-1         Up 12 seconds             8080/tcp
app-2         Up 13 seconds             8080/tcp
balanceador   Up 12 seconds             80/tcp
db-primary    Up 26 seconds (healthy)   5432/tcp
db-replica    Up 23 seconds (healthy)   5432/tcp
gateway       Up 11 seconds (healthy)   0.0.0.0:8000->8000/tcp
```

Solo el **gateway** publica un puerto hacia el computador (`0.0.0.0:8000`). El resto solo existe dentro de la red de Docker.

1. Las peticiones llegan por el gateway y no directamente a las instancias

Por el gateway (puerto 8000) se puede acceder:

```
curl.exe -i http://localhost:8000/api/v1/stock
HTTP/1.1 200 OK
X-Instance: app-1
```

Directo a una instancia (puerto 8080) **no** se puede acceder, porque ese puerto no está publicado:

```
curl.exe -m 3 http://localhost:8080/
```

```
curl: (7) Failed to connect to localhost port 8080 ... Could not connect to server
```

2. El balanceador alterna entre las instancias (cabecera X-Instance)

```
curl.exe -s -i http://localhost:8000/api/v1/stock    # repetir 4 veces
HTTP/1.1 200 OK   X-Instance: app-1
HTTP/1.1 200 OK   X-Instance: app-2
HTTP/1.1 200 OK   X-Instance: app-1
HTTP/1.1 200 OK   X-Instance: app-2
```

Extra: el balanceador retira una instancia caída. Apagamos app-1:

```
docker compose stop app-1
curl.exe -s -i http://localhost:8000/api/v1/stock    # repetir 4 veces
HTTP/1.1 200 OK   X-Instance: app-2
HTTP/1.1 200 OK   X-Instance: app-2
HTTP/1.1 200 OK   X-Instance: app-2
HTTP/1.1 200 OK   X-Instance: app-2
```

Ninguna petición falló: NGINX reintenta en la otra instancia (`proxy_next_upstream`) y saca a app-1 de la rotación durante 10 s (`max_fails=1 fail_timeout=10s`). Para volver a encenderla:

```
docker compose start app-1
```

3. Funciones del gateway: un caso que pasa y otro que se rechaza

3a. API key para escribir (`key-auth`)

Sin API key se **rechaza**:

```
curl.exe -i -X POST http://localhost:8000/api/v1/stock -d "{}"
HTTP/1.1 401 Unauthorized
  "message":"No API key found in request"
```

Con la API key del POS (valor de `API_KEY_POS` en `.env`) **pasa**:

```
curl.exe -i -X POST http://localhost:8000/api/v1/stock -H "apikey: cambiar_esto" -d
"{}"
HTTP/1.1 201 Created
X-Instance: app-1
```

3b. Límite de peticiones para lecturas (`rate-limiting`)

El límite de la demostración es de 10 peticiones por minuto por IP (bajo a propósito; en producción serían 60). En una ráfaga, después de las 10 que permite el límite se **rechaza** con 429:

```
curl.exe -s -o NUL -w "%{http_code} " http://localhost:8000/api/v1/stock   # repetir
10 veces
200 200 200 200 200 200 429 429 429 429
```

(Las 4 peticiones del punto 2 ya habían usado parte del límite de ese minuto.)

```
{ "message":"API rate limit exceeded" }
```

4. La réplica sigue a la primaria

La primaria ve a la réplica conectada por streaming:

```
docker compose exec db-primary psql -U app -d farmacias -c "SELECT client_addr,
state, sync_state FROM pg_stat_replication;"
 client_addr |   state   | sync_state
-------------+-----------+------------
 172.19.0.3  | streaming | async
```

La réplica está en modo recuperación (solo lectura):

```
docker compose exec db-replica psql -U app -d farmacias -c "SELECT
pg_is_in_recovery();"
 pg_is_in_recovery
-------------------
 t
```

Se **escribe en la primaria**...

```
docker compose exec db-primary psql -U app -d farmacias -c "INSERT INTO local
(nombre, comuna, direccion) VALUES ('Farmacia Ñuñoa', 'Ñuñoa', 'Irarrázaval 500')
RETURNING id, nombre;"
 id |     nombre
----+----------------
  4 | Farmacia Ñuñoa
INSERT 0 1
```

...y el dato **aparece en la réplica**:

```
docker compose exec db-replica psql -U app -d farmacias -c "SELECT id, nombre FROM
local WHERE nombre = 'Farmacia Ñuñoa';"
 id |     nombre
```

```
----+----------------
  4 | Farmacia Ñuñoa
```

La réplica **no acepta escrituras**:

```
docker compose exec db-replica psql -U app -d farmacias -c "INSERT INTO local
(nombre, comuna, direccion) VALUES ('x','x','x');"
ERROR:  cannot execute INSERT in a read-only transaction
```